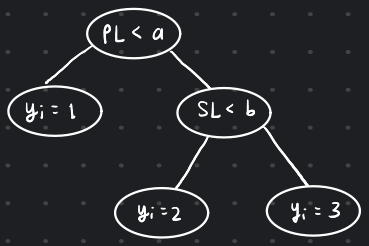
॥ श्रीगणेशाय नमः ॥

Decision Trees

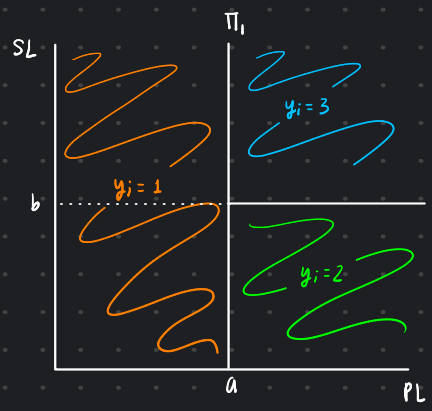
Think of these as nested if...else conditions.
E.g. Consider the Iris dataset:

$x_i = \langle SL, PL, SW, PW \rangle$ $\quad y_i \in \{1, 2, 3\}$

```
if PL < a:
    y_i = 1
else:
    if SL < b:
        y_i = 2
    else:
        y_i = 3
```



All leaves are class labels,
all non-leaves are decisions.



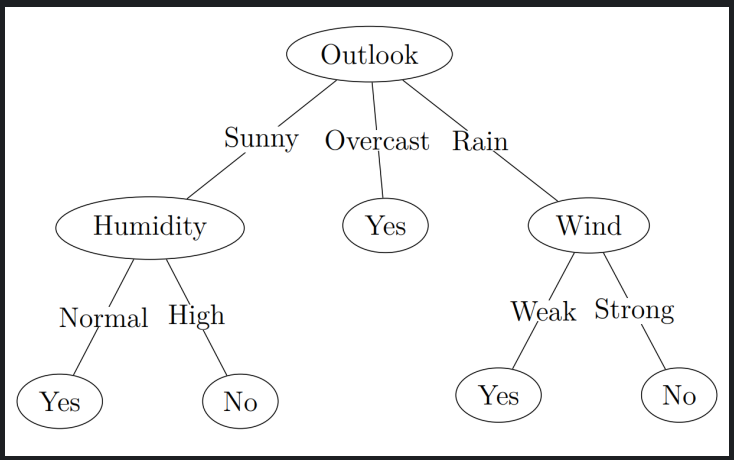
We get a hyperplane corresponding to each decision.

(Axis-parallel hyperplanes)

A decision tree is a set of axis-parallel hyperplanes that divide a region into hypercuboids.

Example: PlayTennis Dataset

Day	Outlook	Temperature	Humidity	Wind	PlayTennis
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No



Here, all the features are categorical, the classification is binary.

Entropy

Consider a random variable Y that can take k values, say y1, y2, ..., yk, then entropy of variable Y is

$$H(Y) = -\sum_{i=1}^{k} P(y_i) \log_b(P(y_i))$$

$P(y_i) = P(Y = y_i)$

b = 2 or e (lg or ln)

PlayTennis
No
No
Yes
Yes
Yes
No
Yes
No
Yes
Yes
Yes
Yes
Yes
No

For the PlayTennis variable:
k = 2; P(Yes) = 9/14; P(No) = 5/14;

$$H(Y) = -\frac{9}{14}\lg\left(\frac{9}{14}\right) - \frac{5}{14}\lg\left(\frac{5}{14}\right) = 0.94$$

Properties

$Y \rightarrow \{Y_+, Y_-\}$ (binary classification)

Case 1:

D → $Y_+ \rightarrow 99\%$, D → $Y_- \rightarrow 1\%$

$$H(Y) = -0.99\lg(0.99) - 0.01\lg(0.01) = 0.0801$$

Case 2:

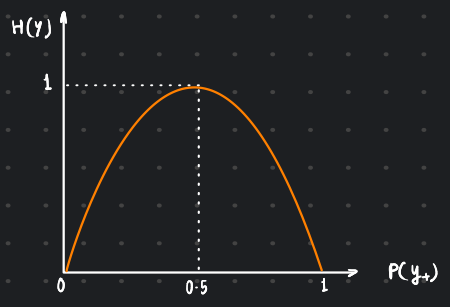
D → $Y_+ \rightarrow 50\%$, D → $Y_- \rightarrow 50\%$

$$H(Y) = -0.5\lg(0.5) - 0.5\lg(0.5) = 1$$

Case 3:

D → $Y_+ \rightarrow 0\%$, D → $Y_- \rightarrow 100\%$

$$H(Y) = 0$$



As the probabilities of both the classes become similar, the entropy increases.

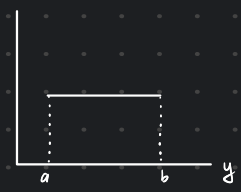
$Y \rightarrow \{y_1, y_2, \ldots, y_k\}$ (k-ary classification)

Entropy is maximum when all the classes are equally probable.

Entropy is minimum when one of the classes is highly likely (P(y1) = 1), and the others are not (P(y2) = ... P(yk) = 0).

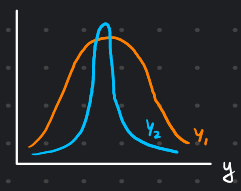
Histogram (or probability density interpretation) of entropy

Say Y has a uniform distribution.



Entropy is maximum.

Say Y1 and Y2 have Gaussian distributions,
Y2 being more peaked than Y1.



Entropy is maximum.

Here Y2 being more peaked means that the probability that Y2 takes a small set of values is really high.
In Y1, the values of more spread.
So Y1 has more equi-probable values.
H(Y2) < H(Y2)

As the Gaussian PDF gets more peaked, the entropy decreases.

Gini Impurity

somewhat similar to entropy

$Y \to \{y_1, y_2, \ldots, y_k\}$

$$I_G(Y) = 1 - \sum_{i=1}^{k} (P(y_i))^2$$

For simplicity, say Y is binary. $Y \to \{y_+, y_-\}$

Case 1:

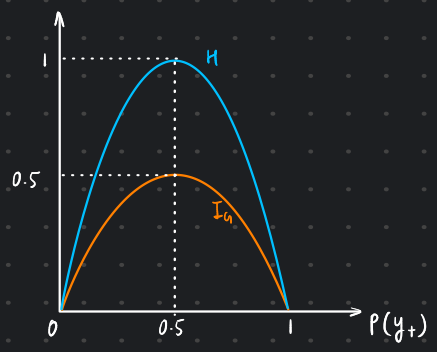
$P(y_+) = 0.5$
$P(y_-) = 0.5$

$I_G(Y) = 1 - (0.5^2 + 0.5^2) = 0.5$
$H(Y) = 1$

Case 2:

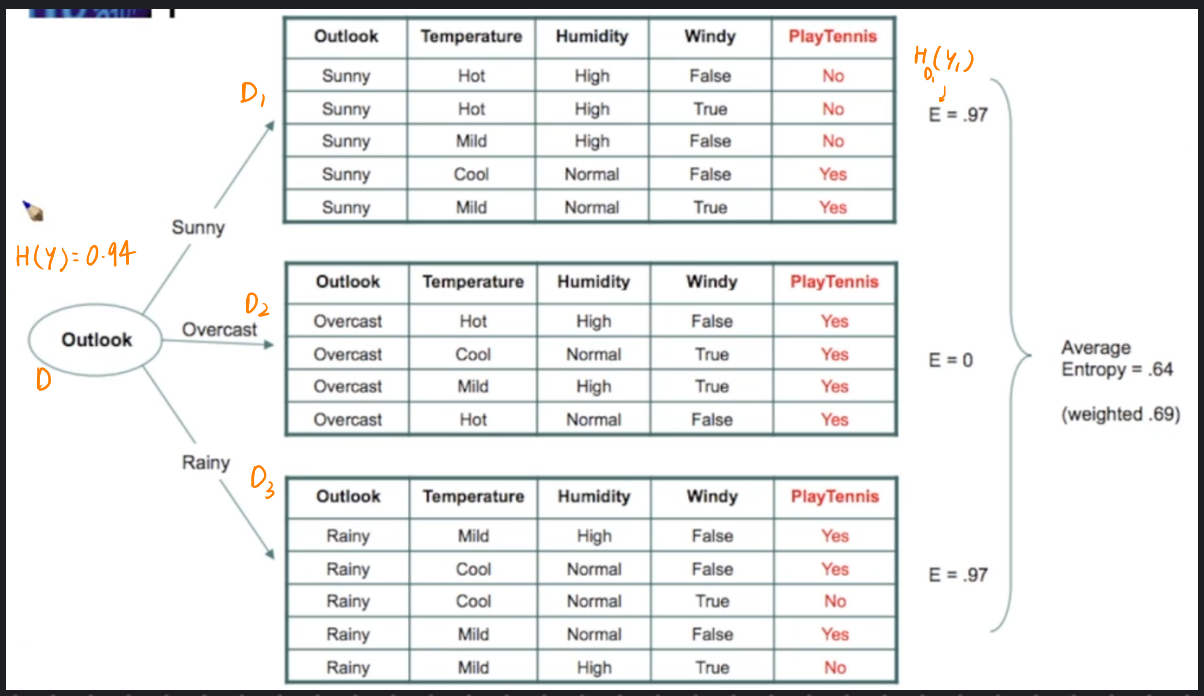
$P(y_+) = 1$
$P(y_-) = 0$

$I_G(Y) = 1 - (1^2 + 0^2) = 0$
$H(Y) = 0$



Both of them behave similarly, but are scaled differently.
Logarithms are computationally more expensive,
so calculating entropy is more expensive as well.

Gini impurity is computationally more efficient.



Outlook	Temperature	Humidity	Windy	PlayTennis
Sunny	Hot	High	False	No
Sunny	Hot	High	True	No
Sunny	Mild	High	False	No
Sunny	Cool	Normal	False	Yes
Sunny	Mild	Normal	True	Yes

Outlook	Temperature	Humidity	Windy	PlayTennis
Overcast	Hot	High	False	Yes
Overcast	Cool	Normal	True	Yes
Overcast	Mild	High	True	Yes
Overcast	Hot	Normal	False	Yes

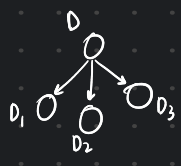
Outlook	Temperature	Humidity	Windy	PlayTennis
Rainy	Mild	High	False	Yes
Rainy	Cool	Normal	False	Yes
Rainy	Cool	Normal	True	No
Rainy	Mild	Normal	False	Yes
Rainy	Mild	High	True	No

Information Gain

$$Gain(D, Ak) = Entropy(D) - Entropy(Ak)$$

A_K is attribute over which D is split.
$H(A_K)$ is the weighted average of entropies of children.

$$IG(Y, outlook) = 0.94 - \left[\frac{5}{14}(0.97) + \frac{4}{14}(0) + \frac{5}{14}(0.97)\right]$$
$$= 0.94 - 0.69 = 0.25$$



$Y \xrightarrow[\text{or attr. } A_K]{\text{var}} y_1, \ldots, y_K$ (over $D_1 \ldots D_K$)

$$Gain(Y, var) = H_D(Y) - \sum_{i=1}^{K} \frac{|D_i|}{|D|} \cdot H_{D_i}(Y)$$

Information_Gain = [Entropy(parent)] – [Weighted Average Entropy of child nodes]

Constructing a DT

We follow a top-down approach.

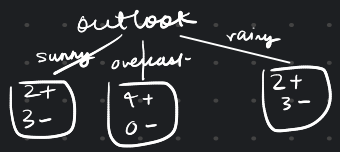
Day	Outlook	Temperature	Humidity	Wind	PlayTennis
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

D → 9 +ve, 5 -ve

$H(Y) = 0.94$

we have 4 features.

Say we choose outlook.



weighted entropy = 0.69

Diff. = 0.94 − 0.69 = 0.25

Gain (Y, outlook) = 0.25

we could've chosen the other 2 features as well.

Temperature → h: 2+, 2- ; m: 4+, 2- ; c: 3+, 1- Gain(Y, temp)

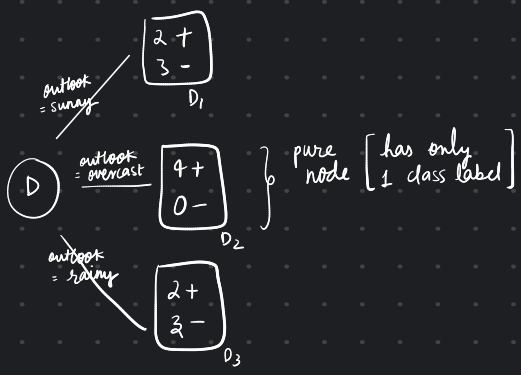
Humidity → H: 3+ ; N: 3- Gain(Y, humidity)

To choose root node, we have

Gain(Y, outlook), Gain(Y, temp), Gain(Y, humidity), Gain(Y, windy)

↳ we pick the feature with the highest information gain.

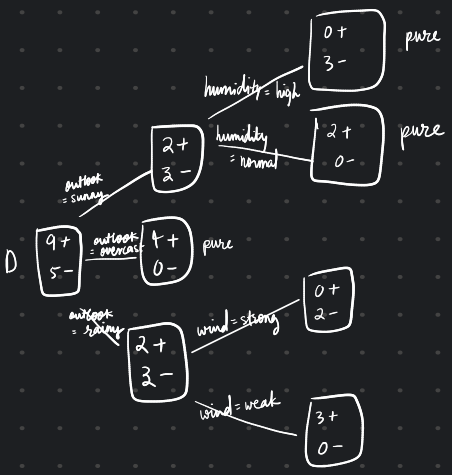
we choose outlook.



pure node [has only 1 class label] ⇒ If outlook = overcast, we'll conclude +ve.

For D_1 → we compute Gain(D_1, humidity), gain(D_1, temp), and gain(D_1, windy)

↳ highest



we recursively break each node using information gain until we reach pure nodes.

3 things can occur:

- pure node → stop growing the tree
- can't grow the tree anymore (lack of points)



→ we then stop growing further. (very few points would be inconclusive. They could simply be noise. 2 points in 10000 could even cause overfitting.)

- if we are too deep → we stop growing.

As depth ↑↑ overfitting ↑↑

In decision trees: hyperparameter = depth

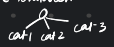
Splitting numerical features

We could use gini impurity in place of entropy (it's computationally efficient).

Say we have numerical features rather than categorical features.

f_1	y
2.2	1
2.6	1
3.5	0
3.8	0
4.6	1
5.5	1

splitting on a categorical feature is straightforward. (A node would have children = # of categories)



For a numerical feature:

① Sort the numerical feature values (say in asc. order)

② Choose threshold:

Some condition like $f_1 < 3.5$

How to decide? We have n possible variations.

We choose various thresholds:

$f_1 < \tau_1$ → D_1, D_2

$f_1 < \tau_2$ → D_1, D_2

⋮

$f_1 < \tau_n$ → D_1, D_2

Each would give a possible pair of data sets.

We check through all possible n splitting criteria.

We choose the one that gives the highest information gain.

Say $f_1 < \tau_2$ has the best IG.

We choose that threshold.

f_1	y
2.2	1
2.6	1
3.5	0
3.8	0
4.6	1
5.5	1

for $\tau = 3.5$

This a cumbersome process. There are hacks around this.